Feature: People, places and skills

From build-plan: feature 12k

Size: heavy - three separate risks with real logic: who can be offered for a service (stored skills, a new rule replacing "nobody ticked means anyone", a data carry-over), a service's own hours (one more layer in the free-time math), and the People and Places pages rebuilt around them.

Status: verified

Branch: feature/12k-people-places-and-skills

Goal

The owner sets her business up the way she thinks of it. A **person** has skills: the services they can do, ticked once on their own card, with their bookable hours beside them. A **service** runs in its own window ("haircuts in the afternoon") and lists only the people who can do it, each with an on/off switch for offering it now. A **place** (a room, a chair) has its own page and shows the services that need it. A customer sees a time only where all of it overlaps: the service runs, a person who can do it is on and free in their hours, and a place it needs is free. Nothing has to be guessed, and nothing bookable today changes when it ships.

Added by Frank on 2026-10-25 after walking Summit Painting (dev): skills were hidden on the service side, a person's hours sat on another page, and people and places shared one page.

In scope

- **Skills on the person.** A person's card on People lists every service with a tick: "Can do". Saving replaces that person's skills. A newly ticked skill also switches them on for that service, so hiring someone is one tick; the service's page switches them off (Frank, 2026-10-25, after the 12k.1 review). A place has no skills.
- **The switch on the service.** A service's "Who does it" lists only the people who can do it, each with On / Off ("offering it now"). A person without the skill is not listed; a line says how to give it ("Tick it on their card on People"). Places it needs are unchanged.
- **The rule.** A service with nobody on cannot be booked: it shows no times and the booking route refuses it. This replaces build-plan item 5's "nobody ticked means anyone can do it". Removing a skill also turns that person off for the service. A save that leaves a service with nobody on still saves, and says so: "Nobody offers Haircut now: customers cannot book it."
- **The carry-over.** One migration, nothing bookable changes: every person ticked on a service today gets that skill, on. Every service with nobody ticked today ("anyone") gives every active person the skill, on. Places ticked today stay as they are.

- **A service's own hours.** On the service form: "Runs: whenever the business is open" or "Its own week", with one-off dates, the same editor a person's hours use. A start time is offered only inside the service's window, the person's bookable hours and the business's open days. A save lists the bookings now outside the window, as a person's hours save does today. Hand bookings are not limited by it, as today.
- **The person's card holds their hours.** Their bookable hours ("Bookable whenever the business is open" / "Own bookable hours", "+ A day that's different") move from Hours to their card on People. Hours keeps only the business's week and its settings.
- **"Booked by customers for"** on each person's card: the services they are on for, or "Not booked by customers".
- **Places get their own page**, Settings, Places ("Rooms, chairs or anything a service needs free"), with its own Add. A place's card ticks the services that need it; that tick is the same one as the service's "Places it needs". People keeps only people.
- **The client setup file**: a service's listed people get the skill and are on; a service that lists nobody gives every person the file lists, and the business's first person, the skill, on.
- **The dev seed** makes skills and switches matching today's ticks.
- Every new word in English and Spanish (12j), and every new API message in the Spanish table.

Out of scope

- Named skills that no service uses (Painting, Crew lead), "Scheduled by you", crews: item 19. Moving these ticks into named skills then is a small data move, paid at item 19 (Frank, 2026-10-25: the owner sees only her own service names).
- Renaming "Services" to "Events" on screen (see Notes).
- Worker sign-in and email replies through the app (later, item 15).
- Any change to the booking window package: it already shows what the API answers.

Build loop

`workflow.stepReview` is `every` and the spec is heavy, so every step gets its own review. After a step's plan has Frank's yes, it runs straight through without asking: build, tests, the browser check, tick, build log, commit and push to `feature/12k-people-places-and-skills`, `/audit`, the independent review. The one planned stop is after the review. Only three things stop a step earlier (AGENTS.md): the plan proves wrong, a line only Frank crosses, or a P0/P1 finding. The new migrations go on live by hand through the tunnel before the merge deploys, on Frank's yes.

Build steps

- ☑ 12k.1 **Skills on the person, the switch on the service.** Migration 0033: table `person_skill`, the carry-over. The free-time and booking rules read "on" people only, and nobody on means not bookable. `PUT /settings/people/:resourceId/skills`; the service's `PUT .../resources` refuses a person without the skill. On People, each person's card gets "Can do"; on Services, "Who does it" lists only those people with On / Off. The setup file and the seed follow. **Done when:** the backend tests prove a person who cannot do a service is never offered, a service with nobody on shows no times and refuses a booking, the carry-over keeps every service's offered people and free times identical on the seeded businesses, and removing a skill turns the person off; on http://localhost:3400 Peter gets "Interior estimate" on his card, appears under it on Services, and switched on is offered in the booking preview.
- ☑ 12k.2 **A service's own hours.** A table `service_hours` (migration 0034; amended while building: `availability_rule` finds the business's row as the one with no person in 26 places, so a service row there would be read as the business's). Free times intersect the service's window with the person's hours. `PUT /settings/services/:serviceId/hours` answers the bookings now outside it. The service form gets "Runs". **Done when:** the tests prove a time outside the service's window is never offered while the same time stays offered for a service without one, a one-off date applies, closed days still close, and a save lists the bookings outside; on the laptop "Interior estimate" set to afternoons offers no mornings in the booking preview.
- ☑ 12k.3 **People and Places pages, the person's card whole.** Places gets its own page and Add; a place's card ticks its services (`PUT /settings/places/:resourceId/services`). A person's card holds their hours, "Can do" and "Booked by customers for"; Hours keeps the business's week only. The words above, in both languages. **Done when:** the route's tests prove a place's ticks are the same rows as the service's "Places it needs"; on the laptop Riverbend Clinic (dev) shows six people and five places on two pages, a person's hours save from their card, and every new word reads in Spanish. (Built: People shows seven, the six practitioners and the business's own first person; places are added and changed through the same `/settings/people` routes as before.)

Files / areas

- `packages/shared/db/`: new `scheduling-tables/person-skill-table.ts` and `service-hours-table.ts` (12k.2); migrations 0033 and 0034; `zod-validation` for the new bodies.
- `backend/lib/scheduling/find-service-resources.ts` (on people only, no "anyone"), `find-free-times.ts`, `backend/lib/bookable-hours/` (the service window, 12k.2), `backend/lib/booking/book-time.ts` and `prepare-booking-move.ts` (refuse a service nobody is on), `backend/lib/settings/` (skills, ticks, service hours, place services), `backend/routes/settings-routes.ts`.
- `packages/shared/client-setup/apply-business-shape.ts`, `packages/shared/scripts/seed-dev.ts`.
- `frontend/components/settings/`: `people-screen.tsx` and a new places screen, the person and place cards, `service-form.tsx` ("Who does it", "Runs"), `hours-screen.tsx` and `person-hours-`

`card.tsx` (moved), `settings-frame.tsx` (Places entry); `frontend/app/(dashboard)/settings/places/`; `frontend/lib/api-client/settings/`; the word files and `spanish-messages.ts`.

Data / contracts

- `person_skill`: `organizationId`, `resourceId`, `bookingLinkId`, `createdAt`; primary key (`resourceId`, `bookingLinkId`); composite foreign keys to `resource` and `booking_link` on (`organizationId`, id), so a skill never crosses businesses; a deleted service takes its skills (cascade). A skill is only ever saved for a person (checked in the save and tested).
- `booking_link_resource` keeps its rows: a person row now means "on", and is allowed only while the person has the skill (the save checks it; removing a skill deletes the "on" row in the same transaction). Place rows are unchanged.
- `service_hours` (12k.2): one row per service, its business and service by composite key, a deleted service takes it; `weeklyHours` null means "whenever the business is open", `dateHours` its one-off dates. No row: it runs whenever the business is open; saving that removes the row.
- `PUT /settings/people/:resourceId/skills { serviceIds: string[] }`: 200 with the person's skills and the services left with nobody on; 400 for a service not the business's or a place; 404 for a person not the business's. Owner and admin only (`CHANGE_BUSINESS`), like every Settings save.
- `PUT /settings/services/:serviceId/resources`: unchanged shape; a person without the skill is a 400 on `peopleIds`: "{name} cannot do this service yet: tick it on their card on People."
- `PUT /settings/services/:serviceId/hours { weeklyHours: WeeklyHours | null, dateHours }`: 200 with the saved hours and the bookings now outside them; the same validation as a person's hours.
- `PUT /settings/places/:resourceId/services { serviceIds: string[] }`: replaces that place's service ticks; 400 for a person.
- `GET /settings/people` gains each person's `serviceIds` (skills) and `onFor` (service ids); places move to `GET /settings/places`. `GET /settings/services` gains each service's hours.
- Free time: a start is offered for a person when the service runs then, the person is on for it and free in their bookable hours, the business is open, and a place it needs is free.

Testing

Backend and shared tests with Vitest against the local `scheduleads_dev` (AGENTS.md): the rule (not able, off, nobody on, removing a skill), the carry-over on the seeded data (same offered people and free times before and after), the setup file both ways, tenant scope on every new route (another business's ids refused), the service window (inside, outside, one-off date, closed day, no window), the bookings-outside list, a place's ticks. The Spanish message test covers the new refusals. The frontend has no test runner; its screens are checked on http://localhost:3400 in English and Spanish.

Notes for the AI

- Frank called services "events" while thinking it through. Screens keep "Services": customers book a "service" in the window (12i), and "event" already means a Google Calendar event in this product. His to overrule.
- Standby (at work, hidden from customers) already exists per person per date; it is not the "on" switch. "On" is per service and lasting; standby is per date and covers every service.
- `booking_link_resource`'s comment ("Nobody ticked means anyone") and build-plan item 5's line change with 12k.1.
- The booking window package needs no change; check its "no times" screen with a service nobody is on.

Implementation walkthrough

Feature 12k was specced heavy, three steps, and kept them: 12k.1 in `807203b` (review fixes in `2d36f3b` and `a945b18`, and the spec amended in `e9b0bf3` with Frank's decision 8: a newly ticked skill switches the person on), 12k.2 in `e7ec89f` (review fixes in `1eef929`), 12k.3 in `398cfb1` (review fixes in `cb25fb6`, `0da782d` and `b0ca08a`). Frank agreed the project plan's decision 26 should state the new rule (`b273b0e`), the spec was marked verified (`735afbb`), and the final review's one point was fixed in `58c8bf5`. The final independent review, over the whole `85c14ce..58c8bf5` delta, passed with nothing new. Migrations 0033 and 0034 were applied to the live database through the tunnel before the merge, on Frank's yes; both live services (IA a la Orden's "Llamada" and "Video llamada") offered the same person before and after.

skills on the person, the switch on the service (12k.1)

Migration 0033 adds `person_skill` (business, person, service; composite foreign keys so a skill never crosses businesses; a deleted service or person takes its skills) and carries today's ticks over: everyone ticked on a service gets the skill, and a service with nobody ticked ("anyone") gives every active person the skill and switches them on, so nothing bookable changed (`skills-carry-over.test.ts` runs the real SQL in a rolled-back transaction). A `booking_link_resource` person row now means "on", allowed only while the person has the skill. `find-service-resources.ts` offers only active people who are on and able; nobody on means no times, and the booking route refuses the service (book by hand follows who is on too, F-490). `PUT /settings/people/:resourceId/skills` replaces a person's skills: a new one switches them on, a removed one switches them off, and the answer names the services left with nobody on. It locks the person `for no key update`, then the changed services `for update` in id order, the same order the Services page's save takes, so the two never deadlock (F-487). The service's `PUT .../resources` refuses a person without the skill ("{name} cannot do this service yet: tick it on their card on People."). The setup file and the dev seed make skills to match.

a service's own hours (12k.2)

The plan put a service's hours in `availability_rule`, but 26 places find the business's row there as the one with no person, so migration 0034 gives them their own `service_hours` table, shaped like a person's row and keyed by the service. `resolve-service-hours.ts` reads it: an own week, or all day (`ALL_DAY_WEEK`) when it runs whenever the business is open, with only its own one-off dates (F-493: whenever open never narrows). `apply-free-times-rules.ts` overlaps each person's windows with the service's on each date (`overlap-windows.ts`); closed days still close; customer bookings and moves re-check through `findFreeTimes`, hand bookings stay free of it. A save lists the bookings now outside the window, as a person's hours do, and a longer length also lists those it pushes past the window (F-494), in time order (F-496). The person's hours card became one shared `own-hours-card.tsx` that a person and a service each bring their words and save to.

People and Places pages (12k.3)

`GET /settings/people` returns only people, each with their skills, what they are on for and their own hours, plus the business's week; `GET /settings/places` returns the places, the services, and the services that need each place. `PUT /settings/places/:resourceId/services` replaces a place's ticks, which are the very `booking_link_resource` rows the Services page writes as "Places it needs"; it locks like the skills save. Places are still added, renamed and switched off through the `/settings/people` routes; each page's Add makes only its own kind. Hours shows only the business's week. On People a closed row says "Booked by customers for: ..." or "Not booked by customers"; on Places, "Needed by: ...". Each person's hours row and each service's "Runs" row sit under their row or form for everyone, the same element whether the form is open or closed, so a Save or Cancel never drops hours not yet saved (F-500, F-502); a member sees them read-only (F-499). New words in English and Spanish ("Reservable siempre que el negocio está abierto", "Horario propio para reservar", "+ Un día distinto", "Lugares"), and the new refusals in the Spanish message table.

decisions and what is carried

- A newly ticked skill switches the person on; the service's page switches them off (decision 8).
- Decision 26 of the project plan now ends: "A person offers a service only while it is ticked on their card and switched on for it; nobody on means customers cannot book it (feature 12k)."
- The overview's fingerprint is computed over the checked-out CRLF form of both plans (F-503).
- Carried, open: F-486 (12j's Calendar views not in that spec), F-498 (unverified: a change of the business's time zone never lists bookings it moves outside a service's own window).
- Not walked in a browser: a member's read-only view (the dev seed has no member login).

Findings

12k/F-487 [P2] closed - A skills save and a "Who does it" save on the same person and service can deadlock, and a customer's booking of that person can be the one aborted

File: backend/lib/settings/save-person-skills.ts:32 **Found:** 2026-10-25 by /audit independent (scope: current, step 12k.1; lens: quality, performance) **Why it matters:** `savePersonSkills` locks the person row `for update` (:32), then the changed services `for update` (:50). `saveServiceResources` locks the service `for update` (save-service-resources.ts:34) and then inserts `booking_link_resource` (:97), whose foreign key check on `booking_link_resource_resource_fk` takes `key share` on the person row, which `for update` blocks. Each then waits on the other. Reproduced with a scratch probe outside the repo on its own throwaway business, running the two saves' exact statements in that order on the local dev database: Postgres answered `40P01 deadlock detected` and aborted the Services save. A booking insert takes `key share` on the service and then on the person (`booking_booking_link_fk` , `booking_person_fk`), so the same cycle can abort a customer's booking of that person while the owner saves the card (reasoned from the lock modes, not reproduced). The comment at :40-41 says the two saves "never cross". The project's own pattern for exactly this case is `save-person-hours.ts:48` , which locks the person `for no key update` with the note "still lets a booking for them be made meanwhile". Rare (both writes inside the same few milliseconds) and the aborted transaction rolls back cleanly, so P2, not P1. **Suggested fix:** Lock the person `.for("no key update")` at :32. It still makes two skills saves of one person run one after the other, and lets the foreign key checks through. The same probe with that one change did not deadlock: the Services save went through and the skills save waited for it. Correct the comment at :40-41. **Resolution:** Fixed 2026-10-25 on feature/12k: the person is locked `for no key update` , as save-person-hours.ts does, so a tick or a booking taking a key share on them while holding the service no longer waits on the skills save. The reviewer's probe: with the old lock `40P01 deadlock detected` ; with the new one the Services save inserts and the skills save waits for the service. Closed 2026-10-25 by /audit independent (re-review of 12k.1 at e9b0bf3): save-person-skills.ts:34 locks the person `for no key update` , then the changed services `for update` in id order (:44-52); save-service-resources.ts:30-34 locks the service first and reads `person_skill` after, so the two saves of one service run one after the other and the skill check sees the committed skills. A fresh scratch probe outside the repo on its own throwaway business (removed after) ran the two saves' statements in the crossing order: with `for update` on the person `40P01` , with `for no key update` the "Who does it" insert went through and the skills save then got the service. A booking takes only `key share` on the person and the service and the skills save never waits on a booking, so no cycle remains. The comment at :32-33 and :42-43 now matches. No new defect.

12k/F-488 [P1] closed - No test books a service nobody is on, though 12k.1's Done when says the tests prove it refuses a booking

File: backend/routes/public-bookings-routes.test.ts (no such test); backend/lib/booking/find-booking-choices.ts:95 **Found:** 2026-10-25 by /audit independent (scope: current, step 12k.1; lens: tests) **Why it matters:** The times route is proved (public-booking-links-routes.test.ts:620: nobody on gives no people and no start times, even when someone can do it). The booking route is not: a POST for a service nobody is on is refused today only because `findBookingChoices` gets an empty candidate list (:95) and answers `time_taken` (:106), and a picked person who is off is refused by `book-time.ts:234` . No test sends either request, so a later change that let an empty list through (for example "any available" falling back to

the business's person, as the old "anyone" did) would pass every suite. The Done when names "refuses a booking" as proven by the backend tests. **Suggested fix:** One route test in public-bookings-routes.test.ts: a service with a person who can do it but is off, and nobody on; POST with nobody picked and with that person picked is refused, and no booking or lead is written. **Resolution:** Fixed 2026-10-25 on feature/12k: public-bookings-routes.test.ts "with nobody on for the service, no booking is taken, for him or for anyone": Sam can do the quote but is off; a booking for him answers 404 and one for anyone 409, and no booking row is written. With the old "anyone" rule put back both answer 201, so the test fails. Fixed again 2026-10-25: the refusal test builds its two forms with the next free time read but left in the list (no painterForm shift), so the later tests keep their eight times. The whole backend suite passes: 94 files, 1085 tests. Reopened 2026-10-25 by /audit independent (re-review of 12k.1 at e9b0bf3), raised to P1: the new test (public-bookings-routes.test.ts:622) proves the refusal, but it builds both requests with `painterForm`, which takes the next of the painter's start times with `painterTimes.shift()` (:559). The painter is open 9:00 to 17:00 with a 60 minute quote and no start interval, so the list holds eight times, and the describe already used all eight. The two times the refused requests take are gone, so "a tick with another number adds its own yes and never moves hers" posts with no `startsAt` and gets 400 instead of 201 at :692. `npm run test --workspace=backend` fails at the target (1 failed, 1084 passed), reproduced on two full runs; the same describe passes with the new test filtered out (5 passed) and the failing test passes alone. Suggested fix: give the two refused requests their own `startsAt` (for example the first of `painterTimes` without shifting it, or `painterTimes.unshift` both back in the `finally`), so nothing the later tests need is used up; then run the whole backend suite. Closed 2026-10-25 by /audit independent (re-review of 12k.1 at a945b18): the refusal test (public-bookings-routes.test.ts:622-659) now builds both requests with `offForm`, which reads `painterTimes[0]` without shifting it, so nothing the later tests need is used up. The time it sends is still a free one, so the 404 for Sam and the 409 for anyone come from nobody being on, not from a taken time, and an emptied list would send no `startsAt` and fail the assertion. `npm run test --workspace=backend` passes in full at the target on two runs (94 files, 1085 tests), including "a tick with another number adds its own yes and never moves hers". The refusal comes before any write (book-time.ts:232-234, find-booking-choices.ts:95-106) and the test checks no booking row is added. No new defect.

12k/F-489 [P3] closed - Ticking a skill on a person's card puts them on for that service at once, so customers can book them, and the spec does not say so

File: backend/lib/settings/save-person-skills.ts:71 **Found:** 2026-10-25 by /audit independent (scope: current, step 12k.1; lens: quality) **Why it matters:** The spec writes down that removing a skill turns the person off, and that the switch on the service is "offering it now"; it never says that adding a skill turns them on. The code does (:71-82, and the card's hint says so), so recording that a trainee can do a live service makes them bookable by customers on the next page load, and the save's notice names only the services left with nobody on, never the ones newly offered. The Done when's "switched on is offered" reads as a separate step. It may well be the intended behaviour; it is a customer-facing decision that lives only in the code and a hint. **Suggested fix:** Write the behaviour into the spec's In scope and Data / contracts (`PUT /settings/people/:resourceId/skills`) so Frank sees it, or, if he prefers, leave a new

skill off until switched on under the service. **Resolution:** Fixed 2026-10-25: Frank chose to keep it (one tick to hire someone; the service's page switches them off), and the spec's "Skills on the person" now says a newly ticked skill switches the person on for that service. Closed 2026-10-25 by /audit independent (re-review of 12k.1 at e9b0bf3): the spec's In scope (current-feature.md, "Skills on the person") says a newly ticked skill switches them on and the service's page switches them off; the code agrees (save-person-skills.ts:73-85 inserts the skill and the "on" row, removing deletes both at :56-72), as do the file comment (:1-3), the route comment (settings-routes.ts:336-338), the card's hint (people-words.ts canDoHint, both languages) and the route test (settings-routes.test.ts: a skills save answers `onFor: [serviceId]`). No new defect.

12k/F-490 [P2] closed - Book by hand still lists a service nobody is on for every person, and the API then refuses it as "not one of this business's"

File: frontend/components/schedule/book-by-hand-form.tsx:48-55 **Found:** 2026-10-25 by /audit independent (re-review of 12k.1 at e9b0bf3; lens: quality) **Why it matters:** The form keeps the old rule: its comment reads "ticked, or nobody ticked, which means anyone", and it lists a live service when `service.peopleIds.length === 0`. Since 12k.1 an empty list means nobody is on, and `bookTime` refuses the person (book-time.ts:232-234, `not_found`), which the schedule route answers as 404 "That service or person is not one of this business's." (schedule-routes.ts:195-198). Every service added since 12k.1 starts with nobody able and nobody on (add-service.ts:45), so it shows in every person's Book by hand, can be the one picked first (`theirs[0]`), and the owner gets a wrong reason after filling the whole form. The form lets the owner pick what the API will refuse. **Suggested fix:** Keep only `service.active && service.peopleIds.includes(personId)` and correct the comment; the existing `noLiveService` line already covers a person on for nothing. **Resolution:** Fixed 2026-10-25: book-by-hand-form.tsx lists a live service for a person only when they are on for it (`peopleIds.includes(personId)`); no other frontend place reads an empty people list as anyone. Closed 2026-10-25 by /audit independent (re-review of 12k.1 at a945b18): book-by-hand-form.tsx:48-53 keeps a service only when `service.active && service.peopleIds.includes(personId)`, matching book-time.ts:234 and find-free-times.ts:75, so the form never offers a service the API refuses for that person, and the comment matches. A search of frontend/ finds no other reader of a service's `peopleIds` that treats an empty list as anyone (service-form.tsx only edits them). No new defect.

12k/F-491 [P3] closed - Two comments still say a service with nobody ticked means anyone

File: packages/shared/db/scheduling-tables/booking-link-resource-table.ts:30-31; packages/shared/zod-validation/admin-validation-schemas/client-setup-validation-schema.ts:59 **Found:** 2026-10-25 by /audit independent (re-review of 12k.1 at e9b0bf3; lens: quality) **Why it matters:** The table's header was updated, but the note on the resource foreign key still gives its reason as "losing the only tick would turn 'only them' into 'anyone'", which 12k.1 made untrue (nobody on now means nobody); the spec's Notes say this table's comment changes with 12k.1. The setup file's `ticked` still says "missing = anyone", while apply-business-shape.ts:268-278 now gives every person the file lists and the first

person the skill, on. A later reader can take the old rule from either. **Suggested fix:** Reword both: the foreign key keeps a person or place from being deleted while ticked; a missing `ticked` gives every person in the file, and the first person, the skill, on. **Resolution:** Fixed 2026-10-25: booking-link-resource-table.ts now says a person on for a service, or a place it needs, cannot be deleted until taken off it; client-setup-validation-schema.ts says no person means every person listed. Closed 2026-10-25 by /audit independent (re-review of 12k.1 at a945b18): booking-link-resource-table.ts:30-31 and client-setup-validation-schema.ts:59 now state the 12k rule, and the latter matches apply-business-shape.ts:268-278 and run-client-setup.ts:251-262. Two other places still carry the old reason; they were not named here and are recorded as F-492.

12k/F-492 [P3] closed - A test comment and the project plan's decision 26 still say a service with nobody ticked means anyone

File: backend/lib/scheduling/booking-link-resource-rules.test.ts:117; blueprint/project-plan.md:589-590 **Found:** 2026-10-25 by /audit independent (re-review of 12k.1 at a945b18; lens: quality) **Why it matters:** F-491 corrected the table's own note, but the test that proves the same foreign key still explains it with "Deleting the only one ticked would leave nobody ticked, which reads as anyone." (:117), and the project plan's decision 26 still ends "Nobody ticked means anyone can do it." (:590), while the build plan's item 5 was changed for 12k.1. Since 12k.1 nobody on means nobody (find-service-resources.ts:74-76), so a later reader of either can take the retired rule as current. Comments and the plan only; no behaviour is wrong. **Suggested fix:** Reword the test comment as the table's note now reads (a person on for a service, or a place it needs, cannot be deleted until taken off it). Decision 26 is in Frank's own plan: amend it the way build-plan item 5 was, with his yes. **Resolution:** Partly fixed 2026-10-25 in e7ec89f: the test comment in booking-link-resource-rules.test.ts now says a delete must never quietly change who is offered. The project plan's decision 26 waits on Frank's yes, so this stays open. Fixed 2026-10-25 (Frank: yes): decision 26 now ends "A person offers a service only while it is ticked on their card and switched on for it; nobody on means customers cannot book it (feature 12k)." Rechecked 2026-10-25 by /audit independent (review of 12k.2 at e7ec89f): booking-link-resource-rules.test.ts:117 now reads "A delete must never quietly change who is offered or which room a service needs.", so the test half is done; blueprint/project-plan.md:589-590 still ends "Nobody ticked means anyone can do it." and waits on Frank. Stays open. Rechecked 2026-10-25 by /audit independent (re-review of 12k.2 at 1eef929): unchanged; blueprint/project-plan.md:589-590 still waits on Frank. Stays open. Closed 2026-10-25 by /audit independent (final review of 12k at 735afbb): b273b0e rewrites decision 26 at blueprint/project-plan.md:589-592 to "A person offers a service only while it is ticked on their card and switched on for it; nobody on means customers cannot book it (feature 12k).", which matches find-service-resources.ts:74-76 and build-plan.md:138-141; booking-link-resource-rules.test.ts:117 still reads as fixed in e7ec89f. `git grep -i "nobody ticked|means anyone|anyone can do"` outside the ledger and history now finds only the spec's own description of the replaced rule, migration 0033's carry-over note and build-plan.md's past tense ("meant"). No new defect in either file; the plan change did leave the overview's fingerprint stale, recorded as F-503.

12k/F-493 [P2] closed - "Runs whenever the business is open" narrows to the business's week once the service has one one-off date, but not without one

File: backend/lib/bookable-hours/resolve-service-hours.ts:42; backend/lib/settings/save-service-hours.ts:93 **Found:** 2026-10-25 by /audit independent (scope: current, step 12k.2; lens: quality, tests) **Why it matters:** A service with no `service_hours` row narrows nothing (resolve-service-hours.ts:31 `if (!row) return null`, the file's own comment "so nothing narrows"). A row with `weeklyHours: null` and one one-off date, which the card shows with the same "Runs whenever the business is open" label, is resolved through `applyBookableHoursRules`, which gives it the business's whole week, so every day of the person's own hours is cut to the business's week. A person's own week replaces the business's (apply-bookable-hours-rules.ts), so it can hold a Saturday or an 8:00 start the business's week does not. A scratch probe outside the repo against the built `backend/dist` rules (business Mon and Tue 9 to 17; person own week Mon 8 to 12 and Sat 10 to 14; a 60 minute service, Nov 2 to 7) gave, with no row, Mon 8, 9, 10, 11 and Sat 10, 11, 12, 13; with a null week and one unrelated one-off date on Dec 1, only Mon 9, 10, 11. So adding one "day that's different" to a service silently removes customer times on every other day, while its row still says it runs whenever the business is open. The save's list has the same split: with no old row `before` is read as the business's week (save-service-hours.ts:93), so when a service gets its own week, a customer booking inside the person's own hours but outside the business's week (the probe's Saturday 10:00) did not "fit before" and is not listed, though it is now outside the service's window. No test has a person whose own week goes past the business's together with a service window. **Suggested fix:** Decide which reading is meant and make both states agree. If "whenever the business is open" means "never narrows", resolve a null week as no weekly narrowing (narrow only on the service's own one-off dates) and read "before" with no row as fitting wherever the person's own hours allow; if it means "inside the business's week", apply the business's week when there is no row too. Add one free-times test and one list test with a person whose own week reaches past the business's. **Resolution:** Fixed 2026-10-25: "whenever the business is open" narrows nothing on an ordinary day, row or no row; only the service's own one-off dates narrow, never the business's. resolve-service-hours.ts and save-service-hours.ts both read a null week as ALL_DAY_WEEK (all-day-week.ts). settings-routes.test.ts "whenever open narrows nothing on an ordinary day, a one-off date saved or not": Rosa's own Saturday stays offered with a Tuesday one-off date saved, and a Tuesday-only week lists Lee's Saturday booking; with the old reading put back the test fails. Closed 2026-10-25 by /audit independent (re-review of 12k.2 at 1eef929): both states now read one way. resolve-service-hours.ts:44-50 hands `applyBookableHoursRules` the row's week or ALL_DAY_WEEK (all-day-week.ts), so as an own week it takes only the service's own one-off dates, never the business's, and on any other day `overlapWindows` with 0 to 1440 leaves the person's windows as they are, the same as no row (find-free-times.ts:77, apply-free-times-rules.ts:53-55). save-service-hours.ts:98-110 reads before and after through the same `window()` (a missing old row is a null week with no dates, so ALL_DAY_WEEK), and `windowsOn` in apply-outside-hours-rules.ts treats it as an own week too, so the list and the free times agree. Closed days still close because the free times take them from the person's hours only. The route test (settings-routes.test.ts:1095) proves Rosa's own Saturday survives a null week with a Tuesday one-off date and that a Tuesday-only week lists Lee's

Saturday booking. No new defect; the stale file header is F-495 and the missing one-off-date read test F-497.

12k/F-494 [P3] closed - A service's new length still lists only the bookings outside the person's hours, never those it pushes past the service's own window

File: backend/lib/settings/save-service.ts:61 **Found:** 2026-10-25 by /audit independent (scope: current, step 12k.2; lens: quality) **Why it matters:** Feature 12d's decision 9 lists the upcoming bookings a new length leaves outside the hours. Since 12k.2 a service has hours of its own, but `saveService` still builds the hours from the business and `findPeopleHours` only (:58-62) and `applyLengthChangeRules` never sees `service_hours`. A service running 9 to 12 with a person on 9 to 17: lengthening an 11:00 booking from 60 to 90 minutes now ends it at 12:30, past the service's window, and the owner is not told. No booking is changed, so only the notice is incomplete. **Suggested fix:** Also check each booking against the service's row (keyed by the service, as save-service-hours.ts does) and list it when it no longer fits either, or record in the spec that a new length checks the person's hours only. **Resolution:** Fixed 2026-10-25: save-service.ts also runs the length-change rule against the service's own window (read as a row keyed by the service), and lists a booking the new length pushes past it. settings-routes.test.ts "a longer length lists a booking it pushes past the service's own window" fails with that check off. Closed 2026-10-25 by /audit independent (re-review of 12k.2 at 1eef929): save-service.ts:74-101 reads the service's row inside the same transaction, which already holds the service `for update` (:33), so a save of its hours (save-service-hours.ts:46, `for no key update` on the same row) cannot change it meanwhile. It runs `applyLengthChangeRules` against that window keyed by the service (a null week as ALL_DAY_WEEK, as the free times read it) and adds only the bookings not already listed, so a booking that did not fit the window before (a hand booking) is still never listed. The route test books 1:00 in a 1:00 to 2:00 window with the business open until 3:00, so only the window can list it. No new defect; the merged list's order is F-496.

12k/F-495 [P3] closed - The header of resolve-service-hours.ts still says a service follows the business's week

File: backend/lib/bookable-hours/resolve-service-hours.ts:1-3 **Found:** 2026-10-25 by /audit independent (re-review of 12k.2 at 1eef929; lens: quality) **Why it matters:** The header reads "resolved like a person's hours: its own week or the business's, with its one-off dates". Since the F-493 fix a service with no week of its own is resolved against ALL_DAY_WEEK, never the business's week, as the comment at :44-45 says correctly. A later reader taking the header at its word could put the business's week back, which is the defect F-493 removed. **Suggested fix:** Say "its own week, or all day when it runs whenever the business is open, with only its own one-off dates". **Resolution:** Fixed 2026-10-25 in 12k.3: the header now says exactly that. Closed 2026-10-25 by /audit independent (review of 12k.3 at 398cfb1): resolve-service-hours.ts:1-3 says its own week, or all day when it runs whenever the business is open, with only its own one-off dates, matching :44-50 (`row.weeklyHours ?? ALL_DAY_WEEK` , `row.dateHours`). No new defect.

12k/F-496 [P3] closed - A new length's list is no longer in time order when both the hours and the service's window list bookings

File: backend/lib/settings/save-service.ts:100-102 **Found:** 2026-10-25 by /audit independent (re-review of 12k.2 at 1eef929; lens: quality) **Why it matters:** `upcoming` comes sorted by start (find-upcoming-bookings.ts orders by `startsAt`) and each of the two results keeps that order, but `both` puts every booking outside the person's hours first and then the ones only past the window. ListedBookings (frontend/components/settings/listed-bookings.tsx) shows them as given, so the owner calling customers can see a Friday booking above a Tuesday one. Every other Settings list is in time order. **Suggested fix:** Keep the order of `upcoming`: filter it by membership in either result, for example by the two sets of booking ids. **Resolution:** Fixed 2026-10-25 in 12k.3: save-service.ts filters `upcoming` by the ids of either list, so it stays in time order and lists each booking once. Closed 2026-10-25 by /audit independent (review of 12k.3 at 398cfb1): save-service.ts:100-102 builds one id set from both results and filters `upcoming` (ordered by `startsAt` in find-upcoming-bookings.ts), so the list is in time order with each booking once, and both results are drawn from that same `upcoming` (:64-98), so nothing outside it can be listed or lost. No new defect.

12k/F-497 [P3] closed - No test reads a service's one-off date from the database into the free times

File: backend/lib/bookable-hours/resolve-service-hours.ts:46-50; backend/routes/settings-routes.test.ts:1130 **Found:** 2026-10-25 by /audit independent (re-review of 12k.2 at 1eef929; lens: tests) **Why it matters:** The rule that a service's one-off date replaces its week is proved only on hand-built hours (apply-free-times-rules.test.ts "the service's one-off date replaces its week that day"). The one route test that saves a one-off date (settings-routes.test.ts:1130) checks only that the Saturday is untouched, never that the Tuesday is narrowed. If `resolveServiceHours` dropped `row.dateHours`, or merged the business's dates in again, every suite would still pass, and that function is exactly where F-493 lived. The Done when's "a one-off date applies" is met at the rules level only. **Suggested fix:** In the same route test, also assert that the Tuesday's free times are only the one-off date's 9:00 to 10:00 start. **Resolution:** Fixed 2026-10-25 in 12k.3: "its own week narrows the times" saves a 9:00 to 10:00 one-off date on the Tuesday over an afternoons week and proves every start that day falls inside it; with `resolveServiceHours` dropping `row.dateHours` the test fails. Closed 2026-10-25 by /audit independent (review of 12k.3 at 398cfb1): settings-routes.test.ts "its own week narrows the times, lists the bookings left outside, and changes none" saves a 9:00 to 10:00 one-off date over an afternoons-only week and asserts the Tuesday has times and every one starts at or after 9:00 and before 10:00; with the date dropped the Tuesday would be afternoons (at or after 13:00) and fail, and with the business's dates merged in nothing it asserts would pass by chance. The test passes in the full backend run. No new defect.

12k/F-499 [P3] closed - A login that cannot change the business no longer sees anyone's bookable hours

File: frontend/components/settings/people-screen.tsx:196-210, :232 **Found:** 2026-10-25 by /audit independent (review of 12k.3 at 398cfb1; lens: quality) **Why it matters:** Before 12k.3 the Hours page showed every active person's hours card to every login, read-only for one without `CHANGE_BUSINESS` (hours-screen.tsx at 1eef929 passed `canEdit={settings.canEdit}` to each PersonHoursCard), under "Your role cannot change the hours. You can see them here." Now the card renders only inside an open Change form (:196-210), and the Change button is shown only when `settings.canEdit` (:232), so a member (the route tests' `helper`) sees the business's week on Hours and nobody's own hours anywhere, while People still tells them "You can see them here." The `canEdit={settings.canEdit}` passed at :206 is therefore always true. The same holds for a place's services on Places (places-screen.tsx:152), which a member cannot see there, though the Services page still shows them. Nothing is changed wrongly; a member loses what they could see. **Suggested fix:** Show a read-only hours line (or the card with `canEdit` false) on each person's row when the login cannot change, or reword People's and Places' `cannotChange` so they no longer promise what is not shown. **Resolution:** Fixed 2026-10-25 (Frank: yes): for a login that may not change the business, each person on People shows their hours card under their row, read-only with no Save (the card Hours showed them before); every place on Places says on its row which services need it ("Needed by: ...", "No service needs it"). Owners still change both through Change. Closed 2026-10-25 by /audit independent (re-review of 12k.3 at 0da782d): people-screen.tsx:80-93 renders PersonHoursCard for every active person in both the open and the closed row (:216, :248) whenever the business has hours, with `canEdit={settings.canEdit}`, which is now false for a member, so OwnHoursCard disables its fieldset (own-hours-card.tsx:130) and shows no Save (:198-204); that is the card the Hours page showed members at 1eef929, active people only, as before. `businessWeek` and `timezone` come from the same `availability_rule` row (find-people-settings.ts:92-93), so a business with hours never hides the card for want of a zone, and one without says "set the business's hours on Hours first" once (:175-177). places-screen.tsx:51-57, :160 says on every closed row which services need it, from the same `needs` map the card's Save updates (:139-141), with a hidden service marked; GET /settings/places returns `placeServices` to every login (settings-routes.ts:310-322). Both "You can see them here." notices are now true. The new words exist in English and Spanish (places-words.ts). No new defect.

12k/F-500 [P3] closed - The card's Save or Cancel silently drops hours edited but not yet saved below it

File: frontend/components/settings/people-screen.tsx:123, :173, :196-210 **Found:** 2026-10-25 by /audit independent (review of 12k.3 at 398cfb1; lens: quality) **Why it matters:** A person's open card is now two forms in one `<li>`: ChangeResourceForm, whose one Save also saves the skills and texts parts through `beforeSave`, and under it PersonHoursCard with its own Save. Saving or cancelling the upper form calls `close()` (:123, :173), which unmounts PersonHoursCard, so a week or a "day that's different" edited there but not saved is thrown away with no notice, while the notice says the person was saved. The skills and texts parts were folded into the one Save so this cannot happen to them. Confirmed from the code path (`setOpen(null)` re-renders the row closed); not walked in a browser. **Suggested fix:** Save the hours through `beforeSave` like the other parts, or keep the upper form from closing (or warn) while the

hours card has unsaved changes; at the least say on the hours card that it has its own Save. **Resolution:** Fixed 2026-10-25 (Frank: yes): a person's hours row now sits under their row or card for everyone, the same element whether the card is open or closed, so the card's Save or Cancel never touches it and its edits stay, marked "Not saved", until its own Save. Checked on http://localhost:3400/settings/people: Mei's hours changed, her card opened and cancelled, the edit still there. "Set the business's hours first" is said once above the list instead of under each row. Closed 2026-10-25 by /audit independent (re-review of 12k.3 at 0da782d): both branches of the row are `<li key={resource.id}>` with two fixed children, [ChangeResourceForm, hoursCardOf(resource)] open (people-screen.tsx:188-217) and [div, hoursCardOf(resource)] closed (:219-249), so when `close()` flips `open` React keeps the same `li` by key and type, replaces only the first child, and keeps the PersonHoursCard in the second slot mounted with its react-hook-form state (own-hours-card.tsx:66-69 reads `defaultValues` once and never resets from props, so the new `person` object each render does not wipe edits). A rename re-sorts `people`, but the keyed `li` moves with its state. The hours card is a sibling of ChangeResourceForm's form, never nested in it. The one case that still unmounts it is the card's Save turning the person off (`resource.active` false, :81), where an off person has no hours to keep, as at 398cfb1. Reasoned from the code; the builder's browser walk on Mei was not repeated here. No new defect.

12k/F-501 [P3] closed - The People read's type comment still says it carries every person and place

File: frontend/lib/api-client/settings/fetch-people-settings.ts:13 **Found:** 2026-10-25 by /audit independent (re-review of 12k.3 at 0da782d; lens: quality) **Why it matters:** The comment over `PeopleSettingsType` reads "Every person and place, on or off, ...", but since 12k.3 GET /settings/people returns people only (find-people-settings.ts:47, `resource.kind` "person"; "Places have their own page") and places come from GET /settings/places. places-screen.tsx imports `ResourceSettingsType` from this file, so a later reader can take the People read as the place to find places. Comment only; no behaviour is wrong. **Suggested fix:** Say "Every person, on or off, with what each can do and their hours, the business's time zone and week, and whether this login may change them." **Resolution:** Fixed 2026-10-25: the comment now says the People read carries every person with what each can do and their hours, the business's time zone and week. Closed 2026-10-25 by /audit independent (final review of 12k at 735afbb): b0ca08a rewrites fetch-people-settings.ts:13-14 to "Every person, on or off, with what each can do and their hours, the business's time zone and week, and whether this login may change them.", which is what GET /settings/people answers (find-people-settings.ts:31-41, people only, `resource.kind` "person" at :47; settings-routes.ts:293 says the same). Places come from fetch-places-settings.ts, which places-screen.tsx reads; it still imports only the shared row type `ResourceSettingsType` from here, which is the same shape for both kinds. Comment only; no new defect.

12k/F-502 [P3] closed - On Services, the "Runs" card under an open service drops its unsaved hours, and the bookings its save listed, when the service's Save or Cancel closes the form

File: frontend/components/settings/services-screen.tsx:72, :128-146, :147 **Found:** 2026-10-25 by /audit independent (final review of 12k at 735afbb; lens: quality) **Why it matters:** A service's ServiceHoursCard is rendered only inside the open row (:128-146, beside ServiceForm); the closed row (:147) is a different `li` with only spans. The service form's Save calls `saved`, which calls `close()` (:72), and its Cancel calls `close()` (:133), so the hours card unmounts: a week or a one-off date edited but not saved is thrown away with no notice, and the list of bookings now outside the window that the card's own save showed (own-hours-card.tsx:64-65, :85, :197, "Stays until the card is saved again or the page is left") is gone, though the spec asks for that list "as a person's hours save does today". F-500 fixed exactly this shape on People (the hours card now sits under the row whether open or closed). Confirmed from the code path (React replaces the second child of the keyed `li` when `open` flips); not walked in a browser. Nothing is saved wrongly. **Suggested fix:** As F-500 did on People: render the service's hours card under its row in both the open and the closed branch (same `li`, same child slot), or hold the outside list at the screen like `held`; at the least keep the form from closing while the hours card is dirty. **Resolution:** Fixed 2026-10-25 (Frank: yes): the "Runs" card now sits under each service's row or form, the same element open or closed, as a person's hours do on People (F-500), so the form's Save or Cancel never drops its edits or the bookings its save listed; a member sees it read-only. Checked on http://localhost:3400/settings/services: Laser's week changed, its form opened and cancelled, the change still there marked "Not saved"; nothing saved. Closed 2026-10-25 by /audit independent (final review of 12k at 58c8bf5): services-screen.tsx:94-104 builds the card once per service through `hoursCardOf`, and both branches of the row are now `<li key={service.id}>` with two fixed children, [ServiceForm, hoursCardOf(service)] open (:141-149) and [div, hoursCardOf(service)] closed (:151-180), so when `saved` (:72) or Cancel (:146) calls `close()`, React keeps the keyed `li`, replaces only the first child, and keeps the ServiceHoursCard in the second slot mounted with its react-hook-form state and its `outside` list (own-hours-card.tsx:65-69 read `defaultValues` once; the card resets only on its own save, :84). Its condition (`businessWeek && timezone`) and props are the same in both branches and never depend on `service.active`, so hiding a service or a rename that re-sorts the list (:69-71, the keyed `li` moves with its state) never unmounts it; opening another service's form flips both rows the same way. The card is a sibling of ServiceForm's `<form>` (service-form.tsx:215-460), never nested, and the "+ Add a service" form renders above the list (:114-120), outside every row; a service it adds gets a card with no row of its own (runs whenever open) once listed. A business with no hours renders no card in either branch, as before. Members now see each card read-only (`canEdit` false disables the fieldset and hides Save, own-hours-card.tsx:130, :198), from `serviceHours` GET /settings/services answers every login (find-services-settings.ts:72-79, settings-routes.ts:186-197), under "You can see them here." Every card's ids and radio names carry the service id (own-hours-card.tsx:100, :146), so the cards now all on the page never collide, and they share `name="service-hours"`, so one opens at a time. Reasoned from the code; the builder's browser walk on Laser was not repeated here. No new defect.

Independent review

Status: passed **Target commit:** 58c8bf52bed95029867a15624376d08c003212f6 **Base commit:** 85c14ce4b92ea98e5c0d6311a52f72cf60534143 **Base ref:** main **Spec hash:**

c1ff9b0da137ae3c1ec2a20f5d829c084b7d608f91147560f8a55d97821398ff **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-25T03:12:11Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-25T03:16:50Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Commands

- `npm run test --workspace=@scheduleads-app/shared` : pass (32 files, 214 tests)
- `npm run test --workspace=backend` : pass (95 files, 1098 tests, local scheduleads_dev)
- `npm run test --workspace=@frankdmosquera/booking-component` : pass (18 files, 98 tests)
- `npm run build --workspace=backend` : pass
- `npm run build --workspace=frontend` : pass
- `npm run lint --workspace=frontend` : pass
- `npm run format:check` : pass

Evidence

- Freshness at start: HEAD 58c8bf5 equals the target, `git merge-base main HEAD` is 85c14ce, the spec's SHA-256 matches, and only findings.md and review.md differ from the target; still so after every command ran.
- 735afbb..58c8bf5 touches only services-screen.tsx and service-hours-card.tsx's header comment: both branches of each service row are `<li key={service.id}>` with the hours card in the same second slot (services-screen.tsx:141-180), so the form's Save or Cancel keeps it mounted with its edits and its outside list (F-502 closed).
- The whole 85c14ce..58c8bf5 delta (102 files) read for the four lenses: migrations 0033 and 0034 and the carry-over, find-service-resources (people on and able and active only), the service window in apply-free-times-rules / resolve-service-hours / overlap-windows, the skills, place-services and service-hours saves (tenant-scoped reads, locks, duplicate ids refused by the schemas), the four new or changed routes (CHANGE_BUSINESS on every save), and the People, Places and Services screens. No focused or skipped test added in the delta.
- F-503 re-checked: the overview marker is still `996777de…` and the plans hash to `be1b829e…` by `/overview`'s contract.

Findings

- F-502 [P3] closed: the Runs card now sits under every service's row or form, same element and slot, so closing the form never drops it.
- F-503 [P3] open: the overview's plan fingerprint is still stale, for `/complete` by its own contract.
- No new findings.

Remaining risk

- F-502's closure and the new read-only cards for members are reasoned from the code; no browser walk of http://localhost:3400/settings/services was repeated by this reviewer (Check not required).
- Every service's Runs card now renders on the Services page at once; not measured on a business with many services (unverified, no profiling).